

Spark Performance Tuning

1. Why Spark Jobs Become Slow

Spark performance problems commonly arise from excessive data movement, poor partition sizing, data skew, inefficient joins, excessive serialization, insufficient executor resources, or inefficient input formats.

A useful troubleshooting process starts by identifying the slow stage rather than immediately increasing cluster size.

2. Partitioning

Partitions determine the units of parallel work. Too few partitions can leave CPUs idle and create large tasks. Too many partitions can create scheduling overhead and excessive small files.

Partition counts should be chosen based on data volume, cluster resources, operation type, and workload characteristics.

3. Repartition vs Coalesce

repartition can increase or decrease the number of partitions and generally causes a shuffle. coalesce is commonly used to reduce partitions with less movement.

```
df2 = df.repartition(200, "customer_id")
df3 = df2.coalesce(20)
```

4. Data Skew

Data skew occurs when some partition keys contain much more data than others. A skewed join may have many fast tasks and a small number of very slow tasks.

Possible techniques include salting, filtering problematic keys, changing join strategy, increasing parallelism, or using Adaptive Query Execution features when appropriate.

5. Broadcast Joins

If one side of a join is small enough, broadcasting can avoid a large shuffle. However, broadcasting a dataset that is too large can cause executor memory pressure.

```
from pyspark.sql.functions import broadcast

result = large_df.join(
    broadcast(small_df),
    on="customer_id",
    how="left"
)
```

6. Adaptive Query Execution

Adaptive Query Execution, commonly called AQE, allows Spark to adjust parts of a physical plan using runtime statistics. Depending on Spark version and configuration, AQE can help with partition coalescing, skew handling, and join strategy changes.

AQE is not a substitute for understanding the workload. It should be evaluated with execution metrics.

7. Caching and Persistence

Caching can help when the same computed dataset is reused multiple times. It can hurt when the cached dataset is too large or rarely reused.

Choose an appropriate storage level and unpersist data that is no longer needed.

8. Memory and Executor Sizing

Executor memory must accommodate user data, execution memory, storage memory, JVM overhead, and workload-specific requirements. Increasing memory blindly may mask a poor partitioning or skew problem.

ExecutorLostFailure and out-of-memory errors should be investigated using logs, stage metrics, garbage collection information, and partition sizes.

9. Small Files

Large numbers of tiny files increase file listing and task scheduling overhead. Compaction and appropriate output partitioning can reduce small-file problems.

Coalescing output before writing can reduce file count, but excessive coalescing can create large tasks.

10. Practical Tuning Checklist

1. Inspect the Spark UI. 2. Identify the slowest stage. 3. Check input and output sizes. 4. Check partition counts and task duration. 5. Look for skew. 6. Inspect joins and shuffles. 7. Check executor memory and GC. 8. Review file sizes. 9. Test the change on representative data. 10. Measure before and after.